

```
/* GameWin – exemple de serveur (Node.js + Express)
   Le SERVEUR garde les points et décide du résultat : le navigateur ne peut plus tricher.
   Démarrage : npm install && JWT_SECRET=un_secret_long node server.js */
const express = require("express");
const cors = require("cors");
const bcrypt = require("bcryptjs");
const jwt = require("jsonwebtoken");
const fs = require("fs");
const crypto = require("crypto");

const SECRET = process.env.JWT_SECRET || "change-moi";
const ORIGIN = process.env.ORIGIN || "https://siriibrahimben-debug.github.io";
const FILE = "data.json";
const STAKES = [10, 25, 50, 100];

let db = fs.existsSync(FILE) ?
JSON.parse(fs.readFileSync(FILE)) : {
  users: {}, bets: {}, ledger: [] };
const save = () => fs.writeFileSync(FILE, JSON.stringify(db, null, 2));

// Chaque mouvement de points est écrit dans un journal (traçabilité)
```

```

function move(user, delta, reason, ref) {
  if (user.points + delta < 0) throw new
Error("Solde insuffisant");
  user.points += delta;
  db.ledger.push({ t: Date.now(), user:
user.pseudo, delta, reason, ref, solde:
user.points });
}

const app = express();
app.use(express.json({ limit: "10kb" }));
app.use(cors({ origin: ORIGIN }));

// anti-abus très simple : 120 requêtes /
minute / IP
const hits = {};
app.use((req, res, next) => {
  const k = req.ip, m =
Math.floor(Date.now() / 60000);
  hits[k] = hits[k] && hits[k].m === m ?
hits[k] : { m, n: 0 };
  if (++hits[k].n > 120) return
res.status(429).json({ error: "Trop de
requêtes" });
  next();
});

function auth(req, res, next) {
  try {
    const t = (req.headers.authorization ||
"").replace("Bearer ", "");
    req.user = db.users[jwt.verify(t,
SECRET).pseudo];
    if (!req.user) throw 0;
  }
}

```

```

    next();
  } catch (e) { res.status(401).json({
error: "Connecte-toi" }); }
}

/* ----- Compte ----- */
app.post("/register", async (req, res) => {
  const { pseudo, password, age18 } =
req.body || {};
  if (!pseudo || pseudo.length < 2 ||
pseudo.length > 20 || !password ||
password.length < 6) return
res.status(400).json({ error: "Pseudo (2-
20) et mot de passe (6+) requis" });
  if (age18 !== true) return
res.status(400).json({ error: "Réservé aux
18 ans et plus" });
  if (db.users[pseudo]) return
res.status(409).json({ error: "Pseudo déjà
pris" });
  db.users[pseudo] = { pseudo, hash: await
bcrypt.hash(password, 10), points: 0,
activeBet: null };
  move(db.users[pseudo], 100, "bonus
inscription");
  save();
  res.json({ token: jwt.sign({ pseudo },
SECRET, { expiresIn: "7d" }), points: 100
});
});
app.post("/login", async (req, res) => {
  const { pseudo, password } = req.body ||
{};
  const u = db.users[pseudo];

```

```

    if (!u || !(await bcrypt.compare(password
    || "", u.hash))) return
    res.status(401).json({ error: "Pseudo ou
    mot de passe incorrect" });
    res.json({ token: jwt.sign({ pseudo },
    SECRET, { expiresIn: "7d" }), points:
    u.points });
  });
  app.get("/me", auth, (req, res) =>
  res.json({ pseudo: req.user.pseudo, points:
  req.user.points }));

  /* ----- Exemple de jeu géré côté
  serveur : « Devine le nombre » -----
  Le nombre secret ne quitte JAMAIS le
  serveur. */
  const LEVELS = [{ max: 100, tries: 8 }, {
  max: 500, tries: 9 }, { max: 1000, tries:
  10 }];

  app.post("/number/start", auth, (req, res)
  => {
    const { stake = 0, level = 0 } = req.body
    || {};
    const u = req.user, L = LEVELS[level];
    if (!L) return res.status(400).json({
    error: "Niveau invalide" });
    if (u.activeBet) return
    res.status(409).json({ error: "Une partie
    est déjà en cours" });
    if (stake !== 0 &&
    !STAKES.includes(stake)) return
    res.status(400).json({ error: "Mise
    invalide" });
  });

```

```

    const id = crypto.randomUUID();
    try { if (stake) move(u, -stake, "mise",
id); } catch (e) { return
res.status(400).json({ error: e.message });
}
    db.bets[id] = { id, user: u.pseudo,
stake, secret: 1 + crypto.randomInt(L.max),
left: L.tries, max: L.max, done: false };
    u.activeBet = id; save();
    res.json({ betId: id, max: L.max, tries:
L.tries, points: u.points });
});

app.post("/number/guess", auth, (req, res)
=> {
    const { betId, guess } = req.body || {};
    const b = db.bets[betId], u = req.user;
    if (!b || b.user !== u.pseudo || b.done)
return res.status(404).json({ error:
"Partie introuvable" });
    if (!Number.isInteger(guess) || guess < 1
|| guess > b.max) return
res.status(400).json({ error: "Nombre
invalide" });
    b.left--;
    let hint = guess < b.secret ? "plus
grand" : "plus petit", won = guess ===
b.secret;
    if (won || b.left <= 0) {
        b.done = true; u.activeBet = null;
        const bonus = b.stake ? Math.max(1,
Math.round(b.stake * 0.1)) : 0;
        if (b.stake) move(u, (won ? b.stake * 2
: 0) + bonus, won ? "mise gagnée + bonus" :

```

```
"bonus fidélité", b.id);
    save();
    return res.json({ finished: true, won,
secret: b.secret, bonus, points: u.points
});
  }
  save();
  res.json({ finished: false, hint, left:
b.left });
});

app.listen(process.env.PORT || 3000, () =>
console.log("GameWin serveur prêt"));
```